Data Cleaning Fundamentals

Master the essential techniques for handling incomplete, noisy, and inconsistent real-world data to ensure quality analysis.



Made with GAMMA

The Data Quality Challenge



Real-world data is rarely perfect. It arrives incomplete, noisy, and inconsistent—posing significant challenges for analysis. Data cleaning routines fill missing values, smooth noise, identify outliers, and correct inconsistencies to prepare datasets for reliable analysis.

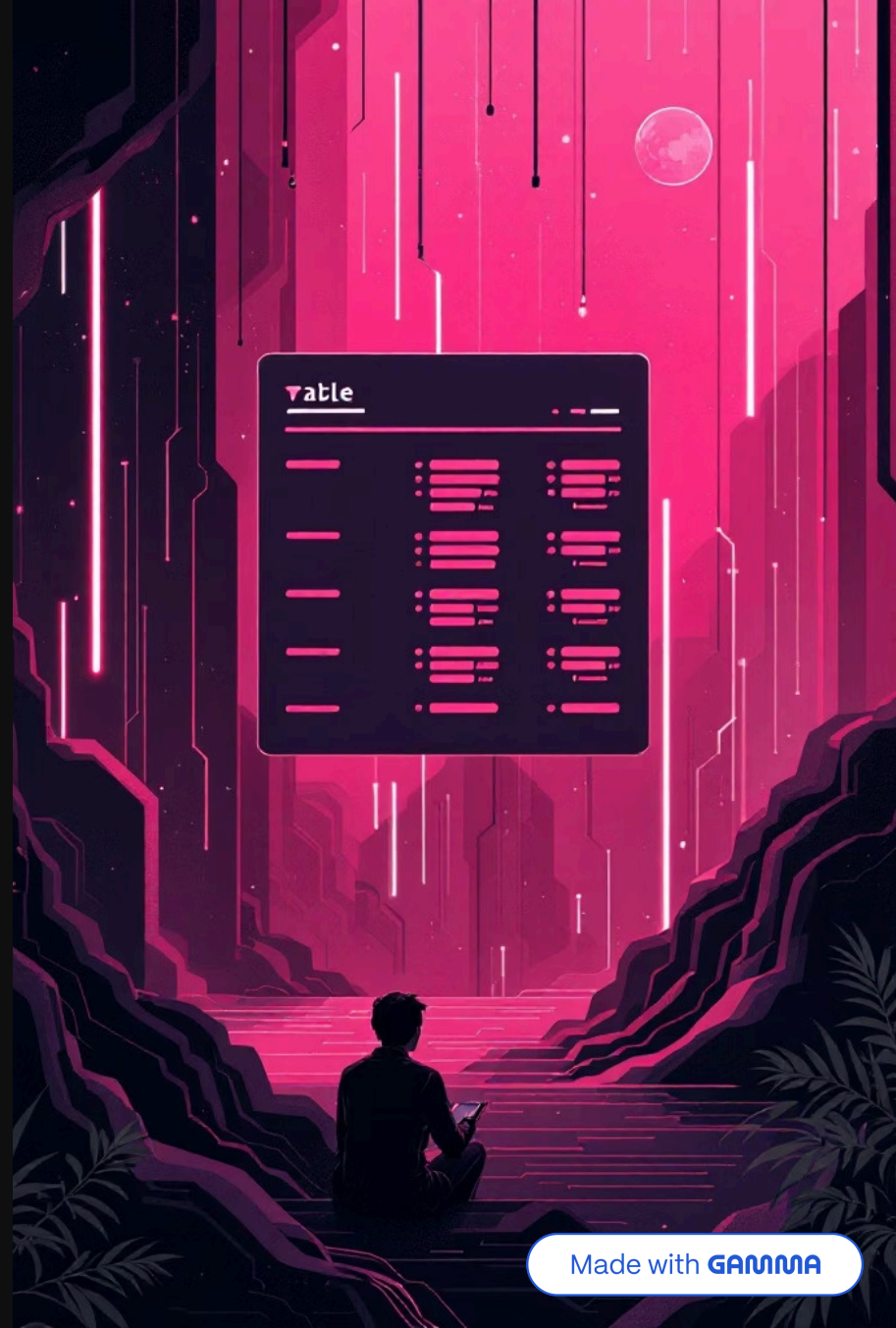
This section explores practical methods for handling missing values, smoothing techniques, and the data cleaning process as a whole.

Made with GAMMA

Understanding Missing Values

When analyzing AllElectronics sales data, you might find customer income fields blank for many records. How do we address these gaps systematically?

Missing values can occur for various reasons—from data entry errors to legitimate cases where information doesn't exist. The key is choosing the right strategy based on your data characteristics and analysis goals.



Made with GAMMA

Method 1: Ignore the Tuple

When to Use

Typically applied when the class label is missing for classification tasks, or when a tuple contains multiple attributes with missing values.

Limitations

Wastes valuable data—ignoring a tuple means losing all other valid attribute values. Poor choice when missing value percentages vary considerably across attributes.

Best Practice

Use sparingly. Only consider when missing values are extensive and other methods aren't feasible.

Made with GAMMA



Method 2: Manual Filling

Simple but Limited

Fill missing values manually by researching or verifying each entry.

Challenges

Extremely time-consuming for large datasets. May not be feasible with thousands of missing values across numerous attributes.

Prone to human error and inconsistency. Best reserved for small datasets or critical fields where accuracy is paramount.

Made with GAMMA

Method 3: Global Constant

Approach

Replace all missing values with a constant like "Unknown" or -1

Advantages

Simple to implement and computationally efficient

Risks

May create false patterns—mining algorithms might interpret "Unknown" as a meaningful category

This method is straightforward but not foolproof. The algorithm may mistakenly identify all "Unknown" values as forming an interesting concept since they share this common label.

Made with GAMMA



Method 4: Central Tendency

Mean vs. Median

Use mean for symmetric (normal) distributions and median for skewed distributions.

Example

AllElectronics customer income is symmetric with mean of $56,000. Replace missing income values with this figure.

This approach leverages statistical measures of central tendency discussed in Chapter 2, providing a reasonable estimate based on data distribution characteristics.

Made with GAMMA

Method 5: Class-Based Central Tendency



Identify Class

Determine the class/category of the tuple with missing values



Calculate Statistics

Compute mean or median for that specific class



Apply Value

Fill missing value with class-specific central tendency

For example, when classifying customers by credit risk, replace missing income with mean income for customers in the same credit risk category. This provides more context-aware estimates than global constants.

Made with GAMMA

Method 6: Most Probable Value

Regression Analysis

Use regression models to predict missing values based on relationships between attributes

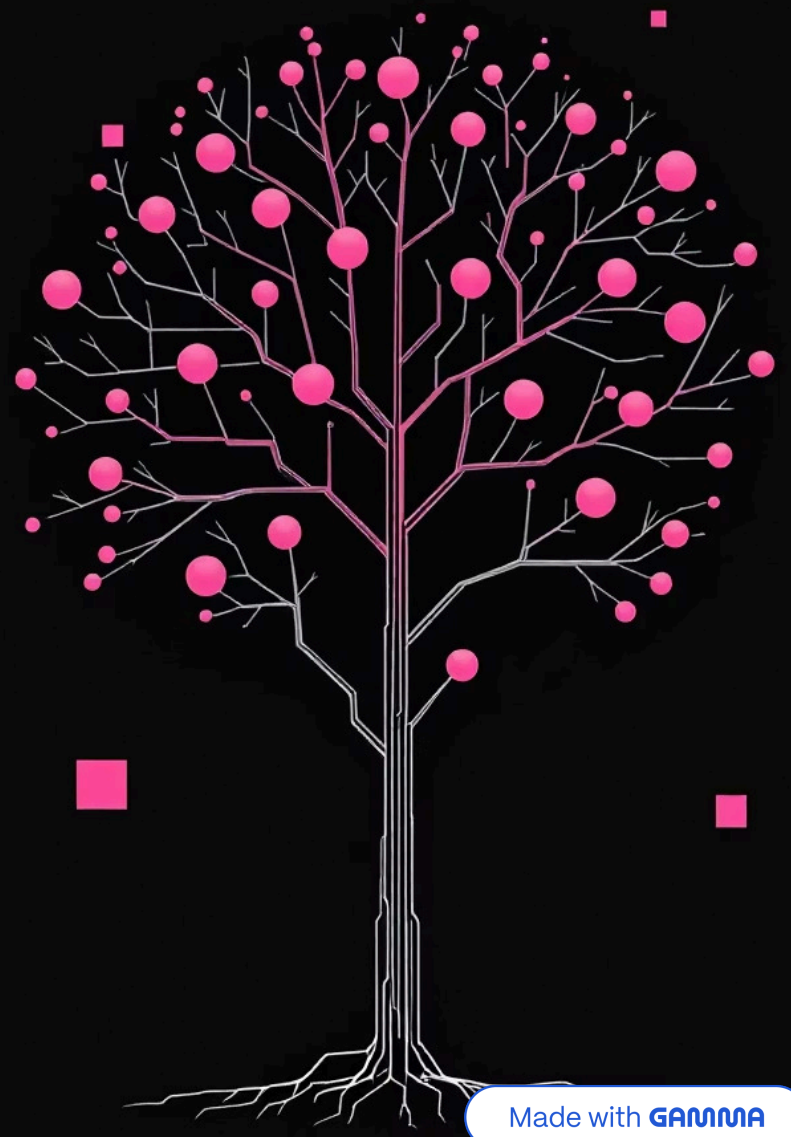
Decision Trees

Construct decision trees using other attributes to predict missing income values

Bayesian Inference

Apply Bayesian formalism for probabilistic inference-based predictions

Most popular strategy—uses maximum available information to predict values, preserving attribute relationships better than other methods.



Made with GAMMA

Key Considerations

Bias Consideration

Methods 3-6 introduce bias—the filled value may not be correct

Not Always an Error

Missing values may be legitimate (e.g., "not applicable" fields)

Prevention Matters

Good database design minimizes missing values from the start

Establish rules for each attribute regarding null conditions. Allow "not applicable" specifications and use software routines to identify null values like "don't know," "?", or "none".

Made with GAMMA